# SIMATS ENGINEERING
## ASSESSMENT TOOL 1

**COURSE NAME: Operating Systems** **COURSE CODE: CSA04**

---

**COURSE OUTCOME COVERED**

***CO5:*** *Analyze the components and functionalities of Linux operating systems and virtualization environments to enhance their operational efficiency*

*Assessment Tool Weightage: 35%*

---

Dr VIMAL V R

**QUESTIONS: 5** **Total Marks:25**

Annexure A

**CONSTRAINT BASED PROBLEM SOLVING**

---

***Code of Conduct:***

*I* **Atchaya Chandran R (192521394)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

# Operating Systems (CSA04)

Assessment Tool 1 — Constraint-Based Problem Solving: Model Answers

Course Outcome CO5: Analyze the components and functionalities of Linux operating systems and virtualization environments to enhance their operational efficiency

Total Marks: 25 | Questions: 5 | Assessment Weightage: 35%

---

*Each answer is organised against the grading rubric — Problem Understanding, Constraint Analysis, Solution Development, Application of Concepts, and Justification — so you can see where each mark comes from. Use this to understand the reasoning and the commands, then write your own submission in your own words before signing the originality declaration on the cover sheet.*

## QUESTION 1 — Linux File System Layout for a 50 GB Server (6 Marks)

*A server with 50 GB total storage must manage rapidly growing system logs, user files, and application data. Design an optimized Linux file-system layout, with partitioning and storage-management tools (e.g. log rotation), so that logs never exceed 10 GB and critical services keep running even under disk pressure.*

### Problem Understanding

The server has a fixed 50 GB envelope that must simultaneously hold the operating system, fast-growing logs, user files, and application data. Left on one undivided partition, any single category — most likely logs, since they are explicitly "rapidly growing" — can expand without bound and silently starve the others of space. The design has to achieve three things at once: separate volatile, fast-growing data from stable system data; enforce a hard 10 GB ceiling on logs; and guarantee that no critical service is denied disk space even if one category fills up.

### Constraint Analysis

- Total capacity is fixed at 50 GB — the layout must be planned up front, not improvised later.
- Logs must never exceed 10 GB — this needs a structural guarantee (partition size), not just an admin policy that can be forgotten or misconfigured.
- User files and application data both grow, but unpredictably, so some flexibility between them is still needed.
- Critical services must survive a "disk full" event in any one category, which is a strong hint that failure domains need to be isolated at the mount-point level, not merely managed by convention.

### Solution Development

**1. Partition layout.** Give each category its own mount point so that one filling up cannot cascade into another. A Logical Volume Manager (LVM) is used across the physical disk so that any of these logical volumes can be grown or shrunk online later, without downtime, as real growth patterns become clear — important given the logs are described as rapidly growing.

| Mount Point | Size | Filesystem | Purpose |
|---|---|---|---|
| /boot | 1 GB | ext4 | Bootloader and kernel images |
| swap | 2 GB | swap | Virtual memory / hibernation |
| / (root) | 8 GB | ext4 | Core OS, binaries, critical service files |
| /var | 4 GB | ext4 | Package cache, spool, non-log variable data |
| /var/log | 10 GB | XFS | Dedicated, hard-capped log volume |
| /home | 15 GB | ext4 | User files |
| /opt | 10 GB | ext4 | Application data |

Because /var/log is its own 10 GB volume, logs physically cannot exceed 10 GB or bleed into /, /home, or /opt — this alone satisfies both the log cap and service-continuity requirements at the architecture level, before any software policy even runs. XFS is used specifically for /var/log because it handles the many small, frequently-appended files typical of syslog rotation efficiently; ext4 remains the safe default elsewhere. Software controls add a second and

third layer of defence on top of this structural guarantee.

**2. Log rotation (software layer 2).** logrotate keeps day-to-day log growth in check and forces an out-of-cycle rotation if any single file balloons intraday:

```
/var/log/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    maxsize 200M
    dateext
    sharedscripts
    postrotate
        systemctl reload rsyslog >/dev/null 2>&1 || true
    endscript
}
```

Rotated logs are compressed and 14 generations are retained; **maxsize** triggers an early rotation if a log grows too fast within a day; the postrotate hook reopens file handles so the logging service never errors out mid-write.

**3. Watchdog script (software layer 3).** A cron job every 15 minutes acts as a last-resort safety net in case rotation alone is not enough:

```
#!/bin/bash
# /usr/local/sbin/logspace-guard.sh
LIMIT_KB=$((10*1024*1024))
USED_KB=$(du -s /var/log | awk '{print $1}')
if [ "$USED_KB" -gt "$LIMIT_KB" ]; then
    find /var/log -name "*.gz" -mtime +3 -delete
    logger -p user.warning "logspace-guard: /var/log exceeded 10GB"
fi
```

**4. Quotas and monitoring.** Disk quotas (quotacheck, edquota) are applied on /home so one user cannot monopolise the 15 GB and starve everyone else. A cron job checks df -h against 80% and 90% thresholds and logs or alerts (via logger, mail, or a monitoring stack such as Zabbix or Prometheus) so an administrator intervenes before any volume actually fills.

## Application of Concepts

The answer applies partition-based failure-domain isolation, LVM for online resizing, filesystem selection matched to workload (XFS for many small appended files), logrotate policy configuration, disk quotas, and threshold-based monitoring — the standard toolbox for capacity-managing a Linux server.

## Justification

The two hard constraints are met independently and redundantly. The log cap is enforced three times over: structurally (a fixed 10 GB volume that cannot grow into anything else), by policy (logrotate's daily rotation and maxsize), and by a watchdog fallback that purges old archives if usage still creeps close to the ceiling. Service continuity holds because the OS and services live on / and /var, which are physically separate mounts from /home and /var/log — exhaustion in one cannot propagate into the other. LVM is chosen over static partitioning specifically because the logs are described as rapidly growing: it allows the layout to be re-balanced online later without backup/repartition/restore downtime.

## QUESTION 2 — Kernel-Level Optimization for Context-Switch Overhead (5 Marks)

*A Linux system under high CPU load suffers frequent context-switching delays. Hardware upgrades are not possible and real-time processing must still be supported. Evaluate kernel-level optimizations — scheduler tuning and kernel modules — to improve performance.*

### Problem Understanding

The system is CPU-bound and losing throughput to context-switch overhead: every switch flushes CPU caches and the TLB, which is pure lost work. The fix must come entirely from software or kernel configuration, since new hardware is off the table, and it must preserve or improve real-time responsiveness — a change that reduces switching frequency at the cost of unpredictable latency for time-critical tasks would violate the requirement rather than satisfy it.

### Constraint Analysis

- No hardware upgrade, so every lever must be a kernel parameter, scheduling policy, or module-level change.
- Real-time support must be preserved, so a single global tweak (e.g. simply lengthening all time slices) is unsafe: it cuts switches but delays latency-sensitive tasks.
- This tension points toward isolating real-time work from the general scheduling pool, rather than one uniform fix applied everywhere.

### Solution Development

- **Scheduler (CFS) tuning** — raising sched_min_granularity_ns and sched_wakeup_granularity_ns increases the minimum time a task runs before it can be pre-empted, cutting the raw switch count for ordinary workloads.
- **Real-time scheduling class** — SCHED_FIFO/SCHED_RR (via chrt) gives latency-critical processes fixed-priority, run-to-block scheduling that pre-empts immediately when runnable, bypassing CFS fairness bookkeeping entirely.
- **CPU isolation + pinning** — the isolcpus boot parameter removes selected cores from the general load balancer, and taskset pins the critical workload there, eliminating switches caused by unrelated task migration onto those cores.
- **IRQ affinity** — hardware interrupts are steered away from the isolated cores, since interrupt handling itself forces involuntary pre-emption.
- **Tickless kernel (NOHZ_FULL)** — removes the periodic timer tick on isolated cores, so a running real-time task is not interrupted on a fixed schedule regardless of need.
- **Kernel module hygiene** — auditing and unloading unused modules removes drivers that register their own interrupt handlers or timers and add background scheduling pressure.
- **cgroup v2 CPU weighting** — a lighter-weight complement to core isolation, guaranteeing the critical workload a protected share of CPU time under contention without physically pinning cores.

```
# A) CFS scheduler granularity (fewer voluntary switches)
sysctl -w kernel.sched_min_granularity_ns=10000000
sysctl -w kernel.sched_wakeup_granularity_ns=15000000

# B) Real-time scheduling class for the critical process
chrt -f 80 /path/to/critical_process

# C) Isolate cores, pin the workload, steer IRQs away
#    (GRUB kernel line): isolcpus=2,3 nohz_full=2,3
taskset -c 2,3 /path/to/critical_process
echo 3 > /proc/irq/44/smp_affinity   # example IRQ, adjust per system

# D) Audit and trim kernel modules
lsmod
rmmod <unused_module>

# E) cgroup CPU weighting
systemctl set-property realtime-app.service CPUWeight=800
systemctl set-property batch-jobs.slice CPUWeight=50

# F) Verify the improvement
vmstat 1 5
perf stat -e context-switches,cpu-migrations -p <PID>
```

If the standard low-latency kernel is still not deterministic enough, the PREEMPT_RT patch makes even kernel critical sections pre-emptible, giving hard real-time guarantees purely through a kernel build change.

### Application of Concepts

This applies CFS scheduler internals, POSIX real-time scheduling classes, CPU and interrupt affinity, tickless-kernel design, cgroup resource control, and kernel module management — a coordinated, layered set of kernel-level tools rather than one isolated fix.

### Justification

Each technique removes a distinct source of switching overhead: granularity tuning cuts voluntary pre-emption, isolcpus removes involuntary migration, IRQ affinity and NOHZ_FULL remove interrupt-induced pre-emption, and RT scheduling with cgroup weighting prevents starvation of the critical task. Combined, total switch volume drops for the bulk of the workload while the real-time task gets deterministic, near-immediate CPU access — both goals met without touching hardware.

## QUESTION 3 — Shell Scripting: 1 Million Log Entries in Under 2 Minutes (5 Marks)

*An administrator must process 1 million log entries per day using only shell scripting, completing within 2 minutes. Design an efficient solution using optimized Linux commands and justify the choice of commands for high performance.*

### Problem Understanding

1,000,000 log entries must be processed daily, entirely with shell scripting, inside a 120-second budget — a required sustained throughput of at least 1,000,000 / 120, roughly 8,333 lines per second. The real risk is not the logic of the processing (filtering, counting, extracting fields) but the execution model: shell scripting makes it easy to write something correct that is nonetheless catastrophically slow.

### Constraint Analysis

- Hard time budget of 120 seconds for 1,000,000 lines means roughly 8,333 lines/sec minimum — line-by-line interpreted loops are the primary risk to eliminate first.
- "Shell scripting only" means relying on standard Linux text-processing utilities rather than a general-purpose compiled program.
- "Optimized Linux commands" is explicit in the question — the choice of tool per operation is itself part of what is being assessed, not just correctness.

### Solution Development

**Why the naive approach fails.** A while read line; do ... done < file loop that calls an external command once per line forks a new process for every one of the 1,000,000 lines. Even at a conservative 1–2 ms of process-creation overhead per fork/exec, that alone costs 1,000–2,000 seconds — over budget before any real work happens. Rule one is therefore: never spawn a process per line.

- **awk** reads and processes the whole file in one pass, in one process, splitting fields and aggregating through associative arrays in O(n) time — typically several hundred thousand lines/sec on ordinary hardware.
- **grep**, especially with fixed strings (-F) or simple extended patterns (-E), uses optimized Boyer–Moore/DFA string search and can exceed a million lines/sec for filtering or counting.
- **sort | uniq -c** is O(n log n); an awk associative array is O(n), so awk is preferred wherever the goal is simple counting rather than needing a sorted key order.
- **LC_ALL=C** switches grep/awk/sort/sed from Unicode-aware collation to plain byte comparison — measurably faster (often 2–3x for sort and grep) and safe when locale-specific ordering is not required.
- Avoid the "useless use of cat" (cat file | cmd); redirecting directly (cmd < file) skips an unnecessary process and pipe.

A single-pass example that counts entries by a field (e.g. severity level in column 4 of a comma-delimited log):

```
export LC_ALL=C
awk -F',' '{count[$4]++} END {for (k in count) print k, count[k]}' access.log
```

If the file is large enough to want extra headroom, splitting across every CPU core scales throughput close to linearly with core count:

```
export LC_ALL=C
N=$(nproc)
split -n l/$N access.log /tmp/chunk_

for f in /tmp/chunk_*; do
    awk -F',' '{c[$4]++} END{for(k in c) print k","c[k]}' "$f" > "$f.out" &
done
wait

cat /tmp/chunk_*.out | \
awk -F',' '{c[$1]+=$2} END{for(k in c) print k, c[k]}' > final_summary.txt
```

split -n l/$N divides the file into N roughly equal chunks without breaking a line in the middle; the loop launches one awk per chunk in the background; wait blocks until every chunk finishes; a final, tiny awk merges the N partial

counts.

### Application of Concepts

This applies process-creation cost versus in-process looping, single-pass O(n) aggregation versus O(n log n) sorting, locale/collation overhead, efficient I/O redirection, and coarse-grained parallelism through split and background jobs — choosing each command for its algorithmic and execution-model properties, not only its functional correctness.

### Justification

A single awk pass alone typically manages at least 200,000 lines/sec, so 1,000,000 lines finish in roughly 5 seconds — more than 20 times inside the 120-second budget, leaving comfortable headroom for real-world disk I/O and heavier per-line logic without needing the parallel variant at all. The parallel/split version is kept in reserve as a scalable option if daily volume grows well past a million lines. The while-read-plus-external-command alternative is rejected outright because per-line process forking alone can exceed the entire time budget before any actual processing occurs.

## QUESTION 4 — Process Management for a Faulty Process at 90% CPU (5 Marks)

*Multiple background services are running; one faulty process consumes 90% of CPU. Without restarting the system, and while keeping critical services unaffected, propose a process management strategy to identify, monitor, and control the faulty process.*

### Problem Understanding

A specific process is monopolising 90% of CPU while other background services keep running. The task has three explicit deliverables — identify, monitor, and control — under two hard constraints: no system restart, and no disruption to other, critical services. That points toward reaching for the least invasive tool first and escalating only if it proves necessary.

### Constraint Analysis

- No reboot, so every action has to be applied to the live system.
- Critical services must stay unaffected, so any fix should be selective — targeting only the faulty process — and ideally give a positive guarantee to the others rather than just assuming they will be fine.
- "Identify, monitor, control" is a three-stage requirement, not a single kill command — a strategy is expected, with escalation only where justified.

### Solution Development

**Stage 1 — Identify.** ps sorted by CPU gives an immediate snapshot and the PID; pidstat sampling over several seconds confirms the usage is sustained rather than a brief spike, and cross-checking against systemd reveals whether the process belongs to a managed unit, which affects how disruptive any fix might be.

**Stage 2 — Diagnose before acting.** A brief strace summarises which system calls the process is spending time in (a tight retry loop versus genuine CPU-bound work), and the unit's recent logs often show the triggering error — this avoids guessing at a fix.

**Stage 3 — Control, least invasive first.**

- Lower its scheduling priority with renice so the CFS scheduler favours every other runnable process under contention — non-disruptive and fully reversible.
- Cap its CPU share directly with cpulimit, which throttles the process via SIGSTOP/SIGCONT to an approximate ceiling without ever terminating it.
- Apply a kernel-enforced quota through systemd's CPUQuota property on its cgroup — the strongest option, since the kernel itself guarantees the limit and it persists for the life of the unit.
- Pin the process away from the cores that critical services use with taskset, removing CPU contention at the hardware level rather than only at the scheduler level.
- Only once diagnosis shows the process is genuinely hung, send SIGTERM first for a graceful exit (letting systemd's Restart=on-failure bring it back cleanly), and reserve SIGKILL for a process that ignores SIGTERM entirely.

```
# Identify
ps aux --sort=-%cpu | head -n 10
pidstat -u 1 5

# Diagnose
strace -c -p <PID>
journalctl -u <service> --since "-10min"

# Control (least to most invasive)
renice -n 19 -p <PID>
cpulimit -p <PID> -l 20
systemctl set-property <service>.service CPUQuota=20%
taskset -cp 0,1 <PID>
kill -SIGTERM <PID>      # graceful, try first
kill -SIGKILL <PID>      # last resort only
```

**Stage 4 — Prevent recurrence.** CPUQuota and MemoryMax are written permanently into the unit file, and a lightweight monitor (a cron job around ps/pidstat, or Prometheus node_exporter with an alert rule) catches the same pattern automatically next time.

## Application of Concepts

This applies Linux process states and scheduling priority (nice/renice), signal handling (SIGTERM versus SIGKILL), cgroup resource isolation, CPU affinity, and systemd unit-level resource control — combined into an escalation ladder rather than one command.

## Justification

The renice, cpulimit, CPUQuota, and taskset controls all act on the offending process alone and are fully reversible, which satisfies “without restarting the system.” CPUQuota in particular gives a kernel-enforced guarantee, not just a hint, that other cgroups — including critical services — retain their share of CPU regardless of what the faulty process does, directly satisfying “critical services remain unaffected” with a mechanism rather than an assumption. Termination signals are placed last because they are the only irreversible option, used only once diagnosis confirms the process will not recover on its own.

## QUESTION 5 — VM Resource Allocation: 5 VMs on 16 GB of Host RAM (4 Marks)

*A physical server hosts 5 VMs with 16 GB total RAM. Each VM needs a minimum of 2 GB, and the host OS needs 4 GB. Propose an efficient resource-allocation strategy using virtualization concepts so every VM operates smoothly under these constraints.*

### Problem Understanding

Five VMs must run on a host with only 16 GB total RAM, where the host itself needs 4 GB and each VM needs a minimum of 2 GB. The arithmetic leaves very little slack, so a workable strategy has to make the small remaining buffer work hard, since it is the only cushion in the system.

### Constraint Analysis

- Host reservation: 4 GB, non-negotiable, never to be encroached on by VMs.
- Guaranteed VM floor: 5 × 2 GB = 10 GB, which must always be honoured per VM.
- Remaining elastic buffer: 16 – 4 – 10 = 2 GB, shared across all 5 VMs.
- If that 2 GB is statically pre-divided (e.g. a fixed 0.4 GB extra per VM), it sits idle behind whichever VM is not currently busy, while a VM that is genuinely under load gets no benefit from it — a static split wastes the one resource that is actually scarce.

### Solution Development

The core idea is to not statically partition the 2 GB buffer, and instead let the hypervisor move it to whichever VM needs it, at the moment it needs it, using standard virtualization memory-management features.

- **Reservation plus limit, not a fixed allocation** — each VM gets a memory reservation of 2 GB (hypervisor-guaranteed floor, honoured even under full contention) and a higher limit of roughly 2.4 GB (2 GB plus an even share of the buffer), so a VM can burst above its floor only when the hypervisor actually has spare memory to give.
- **Memory ballooning** (e.g. KVM/QEMU virtio-balloon, or the VMware/Hyper-V equivalent) — the hypervisor asks an idle VM's guest OS to reclaim its own least-needed pages and return them, which the hypervisor then hands to a VM under pressure. This is the mechanism that actually moves the buffer around dynamically instead of leaving it stranded.
- **Kernel Samepage Merging (KSM)** — if the 5 VMs run similar or identical guest images, KSM finds identical memory pages across VMs and merges them into a single copy-on-write page, freeing real physical RAM and effectively growing the buffer without adding hardware.
- **Conservative overcommit with monitoring** — the 2 GB is treated as a soft pool rather than promised to any one VM, with host free memory actively watched so an administrator can intervene before real exhaustion, not after.
- **Swap as a last-resort safety net only** — a modest host swap area absorbs a rare simultaneous spike across all 5 VMs without an out-of-memory kill, but this is a margin for an edge case, not a substitute for the reservation/ballooning design.

```
# Monitor host and VM memory pressure
free -h
virsh dommemstat <vm-name>
virsh dommemstat --domain <vm-name> --period 5
```

**Numeric summary:** 4 GB host (fixed) + 5 × 2 GB guaranteed VM reservations (10 GB, fixed) + 2 GB elastic pool distributed on demand via ballooning and KSM = 16 GB, fully accounted for, with every VM still able to reach its 2 GB minimum at all times.

### Application of Concepts

This applies memory overcommitment, reservation-versus-limit semantics, paravirtualized ballooning, page deduplication through KSM, and host-level monitoring — the core hypervisor memory-management concepts, used to make a tight, fixed RAM budget behave elastically.

### Justification

Because five independent workloads rarely peak at exactly the same moment, dynamically reallocating the 2 GB buffer through ballooning (and KSM, where applicable) serves far more real demand than any static split could, while the hard reservation of 2 GB per VM and 4 GB for the host means the guaranteed minimums in the problem statement are never violated, even in the worst case — satisfying “all virtual machines operate smoothly” without requiring any additional RAM.

---

*End of Model Answers — CSA04, Assessment Tool 1, CO5*